Treaps

Árvores de busca aleatorizadas

J.D.

UFABC

20 de outubro de 2026

O problema

Dicionário: manter $S \subset U$ sob

- **busca**(x): $x \in S$?
- **inserção**(x)
- **remoção**(x)

O problema

Dicionário: manter $S \subset U$ sob

- **busca**(x): $x \in S$?
- **inserção**(x)
- **remoção**(x)

Árvores binárias de busca: custo proporcional à *altura*.

- Inserções em ordem crescente $\Rightarrow$ árvore degenerada, altura $n - 1$.

O problema

Dicionário: manter $S \subset U$ sob

- **busca**(x): $x \in S$?
- **inserção**(x)
- **remoção**(x)

Árvores binárias de busca: custo proporcional à *altura*.

- Inserções em ordem crescente $\Rightarrow$ árvore degenerada, altura $n - 1$.

Ideia: usar aleatoriedade para que a árvore se comporte como uma árvore de busca *construída em ordem aleatória*, sem depender da ordem das operações.

Definição

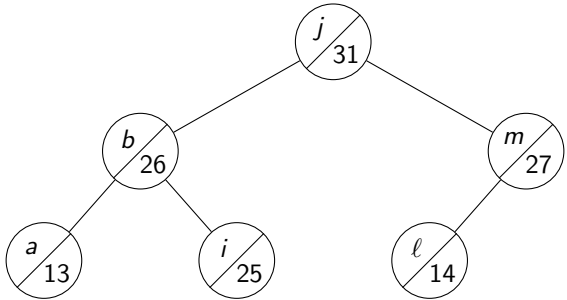
Uma **treap** é uma árvore binária em que cada nó v tem $\mathrm{chave}(v)$ e $\mathrm{prioridade}(v)$ tais que

- é **árvore binária de busca** nas chaves: chaves da subárvore esquerda $<$ $\mathrm{chave}(v)$ $<$ chaves da subárvore direita;
- é **heap de máximo** nas prioridades: $\mathrm{prioridade}(v)$ $>$ prioridade de todo descendente próprio.

Supomos chaves distintas e prioridades distintas.

"Heap" no sentido de árvore ordenada por prioridades (não é uma heap implícita em vetor: a forma não é determinada por completude).

Exemplo



letras = chaves números = prioridades em-ordem: a, b, i, j, ℓ, m

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia (indução no número de nós): a *heap* obriga a raiz a ser o nó de maior prioridade; a busca obriga a subárvore esquerda a ter exatamente as chaves menores que a da raiz e a direita as maiores; cada subárvore é uma treap, única por indução.

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia (indução no número de nós): a *heap* obriga a raiz a ser o nó de maior prioridade; a busca obriga a subárvore esquerda a ter exatamente as chaves menores que a da raiz e a direita as maiores; cada subárvore é uma treap, única por indução.

Consequências

- A forma depende só de chaves e prioridades, *não* da ordem de inserção.
- Equivale a inserir os nós em ordem *decrescente* de prioridade numa ABB vazia.

Rotações

Para consertar *heap* sem estragar a ABB



- A ordem em-ordem A, x, B, y, C é preservada.
- Só a subárvore B muda de pai: custo $O(1)$.

Operações

Busca como em ABB; custo $\propto$ altura.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x

1. busca por x; se falhar, insere x como folha com uma prioridade **sorteada**;
2. enquanto x tem prioridade maior que a do pai y: rotação que sobe x.

A *heap* só pode ser violada entre x e seu pai, e cada rotação preserva a busca.

Rotações $\leq$ profundidade de x.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x
1. busca por x; se falhar, insere x como folha com uma prioridade **sorteada**;
2. enquanto x tem prioridade maior que a do pai y: rotação que sobe x.

A *heap* só pode ser violada entre x e seu pai, e cada rotação preserva a busca. Rotações $\leq$ profundidade de x.

Remoção de x enquanto x não é folha, rotaciona x com o filho de **maior** prioridade (que passa a ser o pai de x e, portanto, vence o outro filho); depois remove a folha. Rotações $\leq$ altura da subárvore de x.

Aleatoriedade

- Prioridade de cada nó: sorteada uniformemente em $[0, 1]$ na inserção, independente das demais (reinserir uma chave sorteia uma prioridade *nova*).
- Com probabilidade 1, as prioridades são distintas.

Aleatoriedade

- Prioridade de cada nó: sorteada uniformemente em $[0, 1]$ na inserção, independente das demais (reinserir uma chave sorteia uma prioridade *nova*).
- Com probabilidade 1, as prioridades são distintas.

Adversário oblívio

A sequência de operações é escolhida *independentemente* das prioridades.
Se o adversário visse as prioridades, poderia remover sempre os nós de menor prioridade, e as restantes deixariam de ser i.i.d. uniformes.

Aleatoriedade

- Prioridade de cada nó: sorteada uniformemente em $[0, 1]$ na inserção, independente das demais (reinserir uma chave sorteia uma prioridade *nova*).
- Com probabilidade 1, as prioridades são distintas.

Adversário oblívio

A sequência de operações é escolhida *independentemente* das prioridades.
Se o adversário visse as prioridades, poderia remover sempre os nós de menor prioridade, e as restantes deixariam de ser i.i.d. uniformes.

Pela unicidade, após qualquer sequência oblívia de operações a árvore é **a treap do conjunto atual, com prioridades i.i.d. uniformes**.
$\Rightarrow$ basta analisar uma treap com n nós e prioridades i.i.d. uniformes.

Profundidade de um nó

Seja x_k o k-ésimo menor elemento de S.

- $x_j \prec x_k$: $x_j \neq x_k$ é *ancestral próprio* de x_k.
- $l_j = \mathbb{1}_{[x_j \prec x_k]}$.
- **Profundidade**: $D_k = \sum_{j \neq k} l_j$ (em arestas).
- **Altura**: $\max_k D_k$.
- $X(j,k) = \{x_j, \ldots, x_k\}$ (para $j < k$; $X(k,j) = X(j,k)$).

Profundidade de um nó

Seja x_k o k-ésimo menor elemento de S.

- $x_j \prec x_k$: $x_j \neq x_k$ é *ancestral próprio* de x_k.
- $l_j = \mathbb{1}_{[x_j \prec x_k]}$.
- **Profundidade**: $D_k = \sum_{j \neq k} l_j$ (em arestas).
- **Altura**: $\max_k D_k$.
- $X(j,k) = \{x_j, \ldots, x_k\}$ (para $j < k$; $X(k,j) = X(j,k)$).

Proposição

*Para $j \neq k$: x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Prova da proposição

Seja a o ancestral comum mais baixo de x_j e x_k (todo nó é ancestral de si mesmo, só nesta prova).

(i) $a \in X(j,k)$: se $a \notin \{x_j, x_k\}$, eles estão em subárvores distintas de a, e a chave de a fica entre as deles.

Prova da proposição

Seja a o ancestral comum mais baixo de x_j e x_k (todo nó é ancestral de si mesmo, só nesta prova).

(i) $a \in X(j,k)$: se $a \notin \{x_j, x_k\}$, eles estão em subárvores distintas de a, e a chave de a fica entre as deles.

(ii) $X(j,k) \subseteq$ subárvore de a: acima de a, x_j e x_k estão do mesmo lado de cada ancestral b, logo qualquer x_i com chave entre as deles também. A busca por x_i segue o caminho de x_j até a.

Prova da proposição

Seja a o ancestral comum mais baixo de x_j e x_k (todo nó é ancestral de si mesmo, só nesta prova).

(i) $a \in X(j,k)$: se $a \notin \{x_j, x_k\}$, eles estão em subárvores distintas de a, e a chave de a fica entre as deles.

(ii) $X(j,k) \subseteq$ subárvore de a: acima de a, x_j e x_k estão do mesmo lado de cada ancestral b, logo qualquer x_i com chave entre as deles também. A busca por x_i segue o caminho de x_j até a.

(iii) Pela *heap*, a tem prioridade maior que a de todos os outros nós da sua subárvore; portanto é o único de maior prioridade em $X(j,k)$.

Prova da proposição

Seja a o ancestral comum mais baixo de x_j e x_k (todo nó é ancestral de si mesmo, só nesta prova).

(i) $a \in X(j,k)$: se $a \notin \{x_j, x_k\}$, eles estão em subárvores distintas de a, e a chave de a fica entre as deles.

(ii) $X(j,k) \subseteq$ subárvore de a: acima de a, x_j e x_k estão do mesmo lado de cada ancestral b, logo qualquer x_i com chave entre as deles também. A busca por x_i segue o caminho de x_j até a.

(iii) Pela *heap*, a tem prioridade maior que a de todos os outros nós da sua subárvore; portanto é o único de maior prioridade em $X(j,k)$.

Como $j \neq k$: $x_j \prec x_k \iff a = x_j \iff x_j$ tem a maior prioridade em $X(j,k)$. $\square$

Profundidade esperada

Prioridades i.i.d. contínuas $\Rightarrow$ todo elemento de $X(j,k)$ é o de maior prioridade com a mesma probabilidade, e $|X(j,k)| = |k-j|+1$:

$$\mathbb{P}[\, l_j = 1 \,] = \frac{1}{|k-j|+1}.$$

Profundidade esperada

Prioridades i.i.d. contínuas $\Rightarrow$ todo elemento de $X(j,k)$ é o de maior prioridade com a mesma probabilidade, e $|X(j,k)| = |k-j|+1$:

$$\mathbb{P}[\, I_j = 1 \,] = \frac{1}{|k-j|+1}.$$

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2.$$

Profundidade esperada

Prioridades i.i.d. contínuas $\Rightarrow$ todo elemento de $X(j,k)$ é o de maior prioridade com a mesma probabilidade, e $|X(j,k)| = |k-j|+1$:

$$\mathbb{P}[\, I_j = 1 \,] = \frac{1}{|k-j|+1}.$$

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2.$$

Com $H_m \leq 1 + \ln m$:

$$\mathbb{E}D_k \leq 2H_n - 2 \leq 2\ln n = O(\log n) \quad \text{para todo } k.$$

Esperança do número de nós no caminho até x_k: $H_k + H_{n-k+1} - 1$.

Alta probabilidade: independência

Queremos Chernoff, que exige independência.

Os I_j *não* são todos independentes

$n = 3$, $k = 2$: $\mathbb{P}[\, x_1 \prec x_2,\ x_3 \prec x_2 \,] = \mathbb{P}[\,$a prioridade de x_2 é a menor$\,] = \frac{1}{3}$, mas $\mathbb{P}[\, x_1 \prec x_2 \,]\mathbb{P}[\, x_3 \prec x_2 \,] = \frac{1}{4}$.

Alta probabilidade: independência

Queremos Chernoff, que exige independência.

Os I_j *não* são todos independentes

$n = 3$, $k = 2$: $\mathbb{P}[\, x_1 \prec x_2,\ x_3 \prec x_2 \,] = \mathbb{P}[\,$a prioridade de x_2 é a menor$\,] = \frac{1}{3}$, mas $\mathbb{P}[\, x_1 \prec x_2 \,]\mathbb{P}[\, x_3 \prec x_2 \,] = \frac{1}{4}$.

Separamos $D_k = L_k + R_k$ com $L_k = \sum_{j<k} I_j$ e $R_k = \sum_{j>k} I_j$.

Lemma

Fixado k, os I_j com $j < k$ são ***independentes****, com $\mathbb{P}[\, I_j = 1 \,] = \frac{1}{k-j+1}$. O mesmo vale para $j > k$.*

Prova do lema

- y_i = prioridade de x_{k-i+1}, $i = 1, \ldots, k$.
- Pela proposição: $I_{k-i+1} = 1 \iff y_i > \max\{y_1, \ldots, y_{i-1}\}$.

Prova do lema

- y_i = prioridade de x_{k-i+1}, $i = 1, \ldots, k$.
- Pela proposição: $I_{k-i+1} = 1 \iff y_i > \max\{y_1, \ldots, y_{i-1}\}$.
- $\rho_i \in \{1, \ldots, i\}$ = posto de y_i entre $y_1, \ldots, y_i$.
- A aplicação permutação $\mapsto (\rho_1, \ldots, \rho_k)$ é injetora (reconstrói-se a permutação inserindo $y_1, y_2, \ldots$ nas posições dadas pelos postos) e os dois conjuntos têm $k!$ elementos: é bijetora.

Prova do lema

- y_i = prioridade de x_{k-i+1}, $i = 1, \ldots, k$.
- Pela proposição: $I_{k-i+1} = 1 \iff y_i > \max\{y_1, \ldots, y_{i-1}\}$.
- $\rho_i \in \{1, \ldots, i\}$ = posto de y_i entre $y_1, \ldots, y_i$.
- A aplicação permutação $\mapsto (\rho_1, \ldots, \rho_k)$ é injetora (reconstrói-se a permutação inserindo $y_1, y_2, \ldots$ nas posições dadas pelos postos) e os dois conjuntos têm $k!$ elementos: é bijetora.
- Logo $\rho_1, \ldots, \rho_k$ são independentes, ρ_i uniforme em $\{1, \ldots, i\}$.
- $I_{k-i+1} = \mathbb{1}_{[\rho_i = i]}$ depende só de ρ_i: independentes, com probabilidade $1/i$. □

Teorema: altura logarítmica

Teorema

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Teorema: altura logarítmica

Teorema

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Em particular, a altura é $O(\log n)$ com probabilidade $1 - O(n^{-7})$, e, nesse instante, cada operação de dicionário tem custo $O(\log n)$ com essa probabilidade.

($n \geq 2$ garante $\ln n > 0$; para $n = 1$ a desigualdade estrita falharia.)

Prova do teorema (1/2)

Fixe k. L_k é soma de $k-1$ Bernoullis independentes de parâmetros $1/i$, $2 \leq i \leq k$, e

$$\mathbb{E}L_k = H_k - 1 \leq \ln n.$$

Prova do teorema (1/2)

Fixe k. L_k é soma de $k-1$ Bernoullis independentes de parâmetros $1/i$, $2 \leq i \leq k$, e

$$\mathbb{E}L_k = H_k - 1 \leq \ln n.$$

Para $s > 0$, usando $1 + x \leq \mathrm{e}^x$:

$$\mathbb{E}\mathrm{e}^{sL_k} = \prod_{i=2}^{k}(1 + \frac{\mathrm{e}^s - 1}{i}) \leq \exp((\mathrm{e}^s - 1)\,\mathbb{E}L_k) \leq \exp((\mathrm{e}^s - 1)\ln n).$$

Prova do teorema (1/2)

Fixe k. L_k é soma de $k-1$ Bernoullis independentes de parâmetros $1/i$, $2 \leq i \leq k$, e

$$\mathbb{E}L_k = H_k - 1 \leq \ln n.$$

Para $s > 0$, usando $1 + x \leq \mathrm{e}^x$:

$$\mathbb{E}\mathrm{e}^{sL_k} = \prod_{i=2}^{k}(1 + \frac{\mathrm{e}^s - 1}{i}) \leq \exp((\mathrm{e}^s - 1)\,\mathbb{E}L_k) \leq \exp((\mathrm{e}^s - 1)\ln n).$$

Markov em e^{sL_k}, com $s = 2$ e limiar $\mathrm{e}^2 \ln n$:

$$\mathbb{P}\big[\, L_k \geq \mathrm{e}^2 \ln n \,\big] \leq \mathrm{e}^{-2\mathrm{e}^2 \ln n}\, \mathrm{e}^{(\mathrm{e}^2-1)\ln n} = n^{-(\mathrm{e}^2+1)} < n^{-8}.$$

Prova do teorema (2/2)

O mesmo vale para R_k. Se $L_k, R_k < \mathrm{e}^2 \ln n$, então $D_k < 2\mathrm{e}^2 \ln n$.

Prova do teorema (2/2)

O mesmo vale para R_k. Se $L_k, R_k < \mathrm{e}^2 \ln n$, então $D_k < 2\mathrm{e}^2 \ln n$.

$$\mathbb{P}\big[\, \exists k : \; D_k \geq 2\mathrm{e}^2 \ln n \,\big] \leq 2n \cdot n^{-8} = 2n^{-7}.$$

Prova do teorema (2/2)

O mesmo vale para R_k. Se $L_k, R_k < \mathrm{e}^2 \ln n$, então $D_k < 2\mathrm{e}^2 \ln n$.

$$\mathbb{P}\big[\, \exists k : \; D_k \geq 2\mathrm{e}^2 \ln n \,\big] \leq 2n \cdot n^{-8} = 2n^{-7}.$$

Fora desse evento, todo nó tem profundidade $< 2\mathrm{e}^2 \ln n < 15 \ln n$ (pois $2\mathrm{e}^2 \approx 14{,}78$).
Altura $< 15 \ln n$ em arestas ($\leq 15 \ln n + 1$ em nós). $\square$

Sequências de operações

O teorema refere-se a uma treap num instante fixo.

Numa sequência oblívia de m operações, seja $N \geq 2$ o maior número de nós ao longo da sequência.

- Cada estado é uma treap com prioridades i.i.d. uniformes.
- Em cada estado $\mathbb{E}L_k \leq \ln n \leq \ln N$: a prova vale com $\ln N$ no lugar de $\ln n$.
- Subaditividade sobre os m estados:

$$\text{todas as operações custam} < 15 \ln N + 1 \text{ com prob. } \geq 1 - 2m\,N^{-7}.$$

- Para $m = O(N^2)$: probabilidade $1 - O(N^{-5})$.

Resumo

- Treap = ABB nas chaves + heap de máximo nas prioridades; **forma única** dado o conjunto de pares.
- Inserção e remoção por **rotações**, $O(1)$ cada.
- Prioridades aleatórias (adversário oblívio) $\Rightarrow$ árvore como ABB de ordem aleatória.
- $x_j \prec x_k \iff x_j$ tem a maior prioridade em $X(j,k)$.
- $\mathbb{E}D_k \leq 2 \ln n$.
- Altura $< 15 \ln n$ com probabilidade $\geq 1 - 2n^{-7}$, via independência de cada lado + MGF/Markov (Chernoff).

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

Árvores binárias de busca (ABB): custo proporcional à *altura*. Inserções em ordem crescente $\Rightarrow$ altura $n - 1$.

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

Árvores binárias de busca (ABB): custo proporcional à *altura*. Inserções em ordem crescente $\Rightarrow$ altura $n - 1$.

Ideia: usar aleatoriedade para que a árvore se comporte como uma ABB *construída em ordem aleatória*, sem depender da ordem das operações.

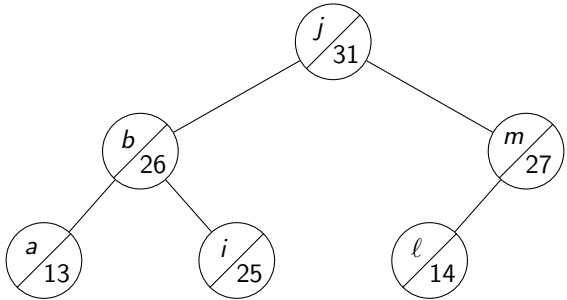
Definição

Uma **treap** é uma árvore binária em que cada nó v tem $\mathrm{chave}(v)$ e $\mathrm{prioridade}(v)$ tais que

- é **ABB** nas chaves;
- é **heap de máximo** nas prioridades: $\mathrm{prioridade}(v) >$ prioridade de todo descendente próprio.

Supomos chaves distintas e prioridades distintas.

Exemplo



letras = chaves números = prioridades em-ordem: a, b, i, j, ℓ, m

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia: a heap obriga a raiz a ser o nó de maior prioridade; a busca determina quais chaves vão para cada lado; aplique indução nas subárvores.

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia: a heap obriga a raiz a ser o nó de maior prioridade; a busca determina quais chaves vão para cada lado; aplique indução nas subárvores.

Consequência: a forma depende só de chaves e prioridades, *não* da ordem de inserção.

Rotações

Consertam a *heap* sem estragar a ABB.



A ordem em-ordem A, x, B, y, C é preservada; custo $O(1)$.

Operações

Busca como em ABB; custo $\propto$ altura.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Remoção de x enquanto x não é folha, rotaciona x com o filho de **maior** prioridade; depois remove a folha.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Remoção de x enquanto x não é folha, rotaciona x com o filho de **maior** prioridade; depois remove a folha.

Número de rotações $\leq$ profundidade (inserção) ou altura da subárvore (remoção) de x: todas as operações custam O(altura).

Aleatoriedade

- Prioridade de cada nó: uniforme em $[0, 1]$, sorteada na inserção, independente das demais.
- **Adversário oblívio**: a sequência de operações é escolhida independentemente das prioridades.

Aleatoriedade

- Prioridade de cada nó: uniforme em $[0, 1]$, sorteada na inserção, independente das demais.
- **Adversário oblívio**: a sequência de operações é escolhida independentemente das prioridades.

Pela unicidade, após qualquer sequência oblívia de operações a árvore é **a treap do conjunto atual, com prioridades i.i.d. uniformes**.
$\Rightarrow$ basta analisar uma treap com n nós e prioridades i.i.d. uniformes.

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Logo $\mathbb{P}[\, x_j \text{ ancestral de } x_k \,] = \dfrac{1}{|k-j|+1}$ e

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2 \leq 2 \ln n.$$

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Logo $\mathbb{P}[\, x_j \text{ ancestral de } x_k \,] = \dfrac{1}{|k-j|+1}$ e

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2 \leq 2 \ln n.$$

Portanto busca, inserção e remoção custam $O(\log n)$ **em esperança**.

Alta probabilidade

Theorem

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Alta probabilidade

Theorem

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Ideia da prova

- $D_k = L_k + R_k$ (ancestrais à esquerda e à direita); cada lado é soma de Bernoullis *independentes* de parâmetros $1/i$.
- Chernoff (MGF + Markov): $\mathbb{P}\big[\, L_k \geq \mathrm{e}^2 \ln n \,\big] < n^{-8}$, e o mesmo para R_k.
- União sobre os n nós.

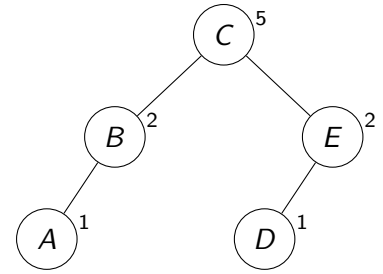
Treap implícita: sequências

- **Sem chave**: a ordem em-ordem *é* a sequência $a_1, \ldots, a_n$.
- Cada nó guarda val, pri e size (nós na subárvore).
- Posição de v na sua subárvore: $\mathrm{size}(\mathrm{esq}(v)) + 1$.
- k-ésimo elemento: desce comparando k com essa posição.
- Inserir no meio não renumera nada.

Treap implícita: sequências

- **Sem chave**: a ordem em-ordem *é* a sequência $a_1, \ldots, a_n$.
- Cada nó guarda val, pri e size (nós na subárvore).
- Posição de v na sua subárvore: $\mathrm{size}(\mathrm{esq}(v)) + 1$.
- k-ésimo elemento: desce comparando k com essa posição.
- Inserir no meio não renumera nada.

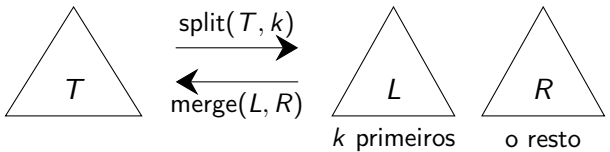
Continua sendo ABB (em posições) + heap (em prioridades): a análise de altura $O(\log n)$ vale igual.



letras = valores números = size

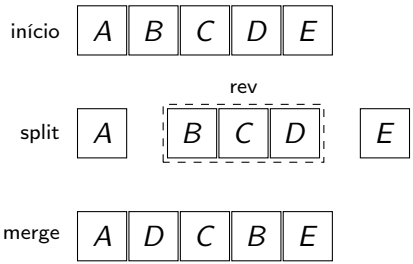
sequência: A, B, C, D, E

Split e merge



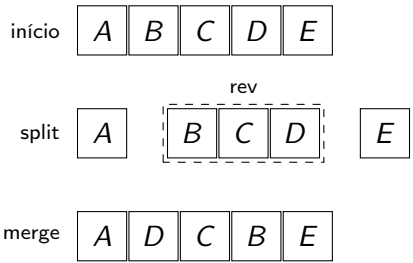
- **split**: desce por *um* caminho, decidindo por size para que lado cada nó vai.
- **merge**: a raiz é a de maior prioridade entre as duas raízes; funde recursivamente o resto. Preserva a heap.
- Ambos custam $O(\text{altura}) = O(\log n)$ esperado.
- Inserir na posição i: $\mathrm{split}(T, i)$, depois dois merge. Remover: dois split e um merge.

Exemplo: reverter as posições 2 a 4



- Dois split isolam M = posições 2 a 4.
- Marca `rev` na raiz de M: nada é trocado ainda.
- Ao descer por um nó marcado, troca-se seus filhos e passa-se a marca a eles (marca *preguiçosa*).
- Dois merge devolvem A, D, C, B, E.

Exemplo: reverter as posições 2 a 4



- Dois split isolam M = posições 2 a 4.
- Marca `rev` na raiz de M: nada é trocado ainda.
- Ao descer por um nó marcado, troca-se seus filhos e passa-se a marca a eles (marca *preguiçosa*).
- Dois merge devolvem A, D, C, B, E.

Mesmo esquema para somar c em um intervalo ou consultar soma/mínimo (agregado em cada nó). Custo $O(\log n)$ esperado por operação.

Resumo

- Treap = ABB nas chaves + heap de máximo nas prioridades; **forma única**.
- Inserção e remoção por **rotações**, $O(1)$ cada.
- Prioridades aleatórias $\Rightarrow$ árvore equivale a ABB de ordem aleatória.
- $\mathbb{E}D_k \leqslant 2 \ln n$.
- Altura $< 15 \ln n$ com probabilidade $\geq 1 - 2n^{-7}$.
- Treap implícita: sequências com *split*/*merge* em $O(\log n)$.